claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a44a930b5bd5d200e.jsonl`
- SHA-256: `bc1bf417204fdbb7d7b709e40be911c3dcea5debf7105a73d51cb4ec4dcd5ab0`
- Source modified: `2026-06-13T19:15:00+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:13:53.200Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"mean_player_motion docstring says 'normalised by the frame width' but the implementation normalises x by width and y by HEIGHT","severity":"low","file":"backend/pipeline/detectors/fusion_features.py:106-123 (and _center_norm at :60-64)","detail":"mean_player_motion's docstring (lines 108-109) claims displacement is 'normalised by the frame width (resolution-invariant)'. But it calls _center_norm, which divides x by frame_width and y by frame_height (lines 63-64). So the per-axis normalisation is anisotropic, not width-only. I verified this: a pure 10px vertical move on a 100x200 frame yields 0.05 (height-normalised), not 0.10 (width-normalised). This is a DOCSTRING/spec mismatch, NOT a math bug ? and the actual behaviour is arguably BETTER than the docstring: I confirmed uniform-scale invariance holds (a 2x upscale of the whole scene gives byte-identical features) and that a 0.1-normalised move in x and a 0.1-normalised move in y both yield 0.1 on both square and 16:9 frames. The only real consequence: the learned-fusion feature is per-axis-normalised, so it is NOT rotation-invariant (a diagonal move's magnitude depends on the W:H aspect ratio). That is fine for a fixed-camera court feature, but the docstring should be corrected so the feature's meaning is not mis-stated to whoever tunes the P3 classifier.","fix":"Reword the mean_player_motion docstring to: 'mean per-track centre displacement between consecutive sampled frames, with x normalised by frame width and y by frame height (per-axis normalised; uniform-scale-invariant, not aspect-ratio-isotropic).' No code change needed unless true isotropy is wanted, in which case normalise both axes by the same dimension."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.129Z)

I'll verify this finding by reading the actual code. Let me start with the key files.

3. user (2026-06-13T19:13:57.060Z)

```
"""Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).

NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
math is unit-testable in a GPU-free dev container. The detection (person boxes,
shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
module only turns those signals into a handful of **scale/translation-invariant**
features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
memorise this court/camera and fail to generalise off a small labelled set).

Conventions:
- A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
- ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
  index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
- A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
- ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
  detection counts ? the player-count-only mode).
- ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).

Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
"""
from __future__ import annotations

from typing import Dict, List, Optional, Sequence, Tuple

BBox = Tuple[float, float, float, float]
Net = Tuple[str, float]


def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
    """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
    (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
    if polygon is None or len(polygon) < 3:
        return False
    x, y = point
    inside = False
    n = len(polygon)
    j = n - 1
    for i in range(n):
        xi, yi = polygon[i]
        xj, yj = polygon[j]
        # Does the horizontal ray at y cross edge (i, j)?
        if (yi > y) != (yj > y):
            x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
            if x < x_cross:
                inside = not inside
        j = i
    return inside


def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
    """Normalised foot point (bottom-centre) of a person box ? where the player stands,
    the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
    if frame_width <= 0 or frame_height <= 0:
        return (0.0, 0.0)
    x1, _y1, x2, y2 = box
```

```
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None."""
        if court_polygon is None:
            return True
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, normalised
        by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
```

```
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
```

```
179             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                    for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                 frame_width: float, frame_height: float,
198                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                 net: Optional[Net] = None,
200                                 min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212
```

4. assistant (2026-06-13T19:14:01.228Z)

The finding hinges on `_center_norm` dividing y by `frame_height`, not `frame_width`. I've confirmed that at lines 60-64. Let me verify the docstring claim at lines 108-109 and check the related `distill_local.FEATURE_NAMES` context to make sure I understand the spec intent. Let me also check the module-level docstring claims about scale-invariance.

5. user (2026-06-13T19:14:02.988Z)

```
df7d5df refactor(detectors): extract the person cue into PersonDetector (fusion P1) (#115)
ecee786 feat(eval): experiment harness ? A/B + SxS variant layer (no-regression by design)
(#114)
9231af2 docs: experiment-harness (A/B + shadow, no-regression) design + clean-slate handoff
(#113)
---STATUS---
?? backend/pipeline/detectors/fusion_features.py
```

6. user (2026-06-13T19:14:04.143Z)

```
backend\pipeline\detectors\fusion_features.py:6:module only turns those signals into a handful
of **scale/translation-invariant**
backend\pipeline\detectors\fusion_features.py:109:    by the frame width (resolution-invariant).
0.0 if <2 frames or no shared tracks."""
backend\pipeline\detectors\tracknet_runner.py:265:        displacement (normalised by frame
width, per second) exceeds
```

7. assistant (2026-06-13T19:14:10.682Z)

Now let me verify the math claim in the finding directly. The finding claims a 10px vertical move on a 100x200 frame yields 0.05 (height-normalised), not 0.10 (width-normalised). Let me trace the code:

- `_center_norm` divides x by `frame_width` and y by `frame_height` (lines 63-64). This is confirmed by reading.
- `mean_player_motion` computes Euclidean distance over these per-axis-normalised centres (line 120).
- A pure 10px vertical move: `cy` changes by `10/200 = 0.05`, `cx` unchanged. Euclidean magnitude = 0.05. The docstring says "normalised by the frame width" which would imply `10/100 = 0.10`. Confirmed mismatch.

Let me confirm there's no test that pins the docstring's literal "width" claim and check the `tracknet_runner.py:265` reference to see if "normalised by frame width" is the established house convention (which would strengthen the doc-mismatch claim by showing inconsistency).

8. user (2026-06-13T19:14:11.631Z)

```
258             merge_gap: float = 2.0,
259             min_window_duration: float = 2.0,
260             chunk_index: Optional[int] = None,
261         ) -> List[Dict[str, Any]]:
262             """Convert a shuttle trajectory into candidate rally windows.
263
264             A frame is "active" when the shuttle is visible AND its inter-frame
265             displacement (normalised by frame width, per second) exceeds
266             ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267             ``merge_gap`` seconds of each other are grouped into one window; short
268             windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269             shape feeding the AI-handoff phase is identical.
270             """
271             if fps <= 0:
272                 fps = 30.0
273             if frame_width <= 0:
274                 frame_width = 1280.0
275
276             # Compute per-frame normalised velocity from the last *visible* point.
277             active = []  # (frame_idx, velocity)
```

9. user (2026-06-13T19:14:13.289Z)

```
tests\test_fusion_features.py:68:# ------------------------------------------------------------- mean_player_motion
tests\test_fusion_features.py:70:def test_mean_player_motion():
tests\test_fusion_features.py:73:    assert ff.mean_player_motion(per_frame, 100, 100) ==
pytest.approx(0.10)
tests\test_fusion_features.py:75:    assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]},
100, 100) == 0.0
tests\test_fusion_features.py:76:    assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1:
[(2, (10, 0, 20, 10))]}, 100, 100) == 0.0
```

10. assistant (2026-06-13T19:14:21.320Z)

Let me read the test to see exactly what it pins, since the test uses a square 100x100 frame where width==height (which would NOT distinguish the two normalization conventions).

11. user (2026-06-13T19:14:21.855Z)

```
68      # ------------------------------------------------------------- mean_player_motion
69
70      def test_mean_player_motion():
71          # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
```

```
72          per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
73          assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
74          # single frame -> 0; no shared track id -> 0
75          assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
76          assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79      # ---------------------------------------------------------------- two_sided_motion
80
81      def test_two_sided_motion():
82          net = ("y", 0.5)
```

12. assistant (2026-06-13T19:14:42.457Z)

The test at line 70-73 uses a square 100x100 frame with a horizontal (x) move, which produces 0.10 under both conventions ? so it does not pin the docstring's "width" claim, and it does not catch the anisotropy. This confirms the finding's analysis: the test is blind to the x-vs-y distinction.

Let me now confirm my own analysis of the math by tracing it one more time carefully, and verify the finding's "uniform-scale-invariant" claim is what actually holds.

Tracing `mean_player_motion` for the finding's hypothetical (10px vertical move, 100x200 frame):
- Frame a: box center y-norm = `y/200`; Frame b: box moved 10px down ? y-norm changes by `10/200 = 0.05`, x unchanged.
- Euclidean magnitude = `sqrt(0 + 0.05²) = 0.05`.
- Under the docstring's stated "normalised by the frame width" (100), it would be `10/100 = 0.10`.

The implementation indeed yields 0.05 (height-normalised on the y-axis), contradicting the docstring's "normalised by the frame width". Confirmed.

Now I have everything needed. Let me also verify the "uniform-scale-invariant" claim the finding makes ? a 2x upscale of the whole scene. If all pixel coords and both frame dims double, each normalized coordinate is identical, so the feature is byte-identical. That holds. And the rotation/aspect-ratio non-isotropy: a diagonal move's magnitude mixes `dx/W` and `dy/H`, which differ when W?H. Correct.

Assessment complete.

The finding is accurate on every checkable point:

1. **Code reality confirmed.** `_center_norm` (fusion_features.py:60-64) divides x by `frame_width` and y by `frame_height`. `mean_player_motion` (lines 106-123) calls it and takes a Euclidean magnitude over those per-axis-normalised centres (line 120). So normalisation is anisotropic (per-axis), not width-only.

2. **Docstring mismatch confirmed.** Line 108-109 literally states the displacement is "normalised by the frame width (resolution-invariant)". That is wrong for the y-axis. The finding's worked example (10px vertical move on 100x200 ? 0.05, not 0.10) is correct.

3. **It is a spec/doc mismatch, not a math bug.** The finding explicitly frames it that way and is right: the per-axis behaviour is uniform-scale-invariant (a 2x scene upscale doubles all pixel coords and both dims, leaving normalised coords identical) but not aspect-ratio-isotropic / rotation-invariant. For a fixed-camera court feature that is acceptable.

4. **The existing test does not catch it.** test_fusion_features.py:70-73 uses a square 100x100 frame with a horizontal move, where both conventions coincide at 0.10 ? so it neither pins "width" nor exercises the x/y asymmetry.

5. **House-convention inconsistency adds weight.** tracknet_runner.py:265 genuinely normalises shuttle velocity by frame width, so "normalised by frame width" is a real convention elsewhere ? which is exactly why copying that phrase into a function that does per-axis normalisation is misleading to a P3-classifier tuner.

Severity is correctly "low": it is a docstring-only wording defect with no runtime impact, and the suggested fix is a pure doc reword (no code change needed unless true isotropy is desired). This is a real, low-severity documentation accuracy issue, not a false positive or pure nitpick (the docstring makes a concrete, falsifiable, and false claim about feature semantics that a downstream tuner relies on).

13. user (2026-06-13T19:14:52.746Z)

Structured output provided successfully

14. assistant (2026-06-13T19:15:00.334Z)

Verified the finding against the actual code. It holds: the `mean_player_motion` docstring at `backend/pipeline/detectors/fusion_features.py:108-109` claims "normalised by the frame width", but `_center_norm` (lines 60-64) divides y by `frame_height`, making it per-axis-normalised. Real low-severity documentation defect; the existing test (square 100x100 frame, horizontal move) cannot catch it. Returned `real=true`, severity `low`, with a doc-reword fix.